

Make `mdspan` `size_type` controllable

Document #: P2553R1
Date: 2022-03-15
Project: Programming Language C++
LEWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@stellarscience.com>
Daniel Sunderland
<dansunderland@gmail.com>

Contents

- 1 Revision History
 - 1.1 R1
 - 1.2 Initial Version 2022-02 Mailing
 - 1.2.1 Review in LEWG 2022-03-01
- 2 Description
 - 2.1 Example
 - 2.2 Possible Ways To Address The Issue
 - 2.2.1 Mappings Doing Offset Calculation With Argument Type
 - 2.2.2 Template extents on `size_type`
 - 2.2.3 Second Extents Type Templated on `size_type`
 - 2.2.4 Template LayoutPolicy on `size_type`
 - 2.2.5 What we prefer:
 - 2.3 Why we can't fix this later
- 3 Editing Notes
- 4 Wording
 - 4.1 In 22.7.X [`mdspan.syn`]
 - 4.2 In 22.7.X.1 [`mdspan.extents.overview`]:

- 4.3 In 22.7.X.2 [mdspan.extents.helpers]:
- 4.4 In subsection 22.7.X.3 [mdspan.extents.cons]
- 4.5 In subsection 22.7.X.4 [mdspan.extents.obs]
- 4.6 In subsection 22.7.X.5 [mdspan.extents.compare]
- 4.7 In subsection 22.7.X.6 [mdspan.extents.dextents]
- 4.8 In subsection 22.7.X.3.1 [mdspan.layout.left.ctors]
- 4.9 In subsection 22.7.X.4.1 [mdspan.layout.right.ctors]
- 4.10 In Subsection 22.7.X.5.2 [mdspan.layout.stride.ctors]
- 4.11 In subsection 22.7.X.1 [mdspan.mdspan.overview]
- 4.12 In subsection 22.7.X.1 [mdspan.mdspan.ctors]
- 4.13 In Subsection 22.7.X [mdspan.submdspan]
- 5 Implementation
- 6 Related Work

§ 1 Revision History

§ 1.1 R1

- don't constrain the index-type via `unsigned_integral` concept
- add preconditions to `extents` constructor to account for signed types
- make `extents` conversion construction explicit when narrowing `size_type`

§ 1.2 Initial Version 2022-02 Mailing

§ 1.2.1 Review in LEWG 2022-03-01

- don't constrain the index-type via `unsigned_integral`
- allow signed integers as `size_type` for `extents`

§ 2 Description

P0009 explicitly sets the `size_type` of `extents` to `size_t`, which is then used by *layout mappings* and `mdspan`. While this matches `span` whose `extent` function returns `size_t`, this behavior has significant performance impact on various architectures where 64-bit integer throughput is significantly lower than 32-bit integer computation throughput.

While that problem is present for `span` it is a much bigger issue for `mdspan`, since every data access into a multi-dimensional array is accompanied by an offset calculation which typically costs two integer operations per dimension.

On current GPUs, which are an essential hardware component in machines used for high performance computing, machine learning and graphics (arguably the core constituency for `mdspan`), the ratio between 32-bit and 64-bit integer throughput is often 2x-4x.

§ 2.1 Example

To gauge the impact, we investigated a simple stencil code benchmark, which is hosted in the [mdspan reference implementation](#). That benchmark is using CUDA and compares a variant with raw pointers and explicit index calculation against a version which uses `mdspan`.

The `mdspan` variant does in each CUDA thread the following code:

```
for(size_t i = blockIdx.x*d; i < s.extent(0)-d; i += gridDim.x) {
    for(size_t j = threadIdx.z*d; j < s.extent(1)-d; j += blockDim.z) {
        for(size_t k = threadIdx.y*d; k < s.extent(2)-d; k += blockDim.y) {
            value_type sum_local = 0;
            for(size_t di = i-d; di < i+d+1; di++) {
                for(size_t dj = j-d; dj < j+d+1; dj++) {
                    for(size_t dk = k-d; dk < k+d+1; dk++) {
                        sum_local += s(di, dj, dk);
                    }
                }
            }
            o(i,j,k) = sum_local;
        }
    }
}
```

The raw pointer variant looks like this:

```
for(size_t i = blockIdx.x*d; i < x-d; i += gridDim.x) {
    for(size_t j = threadIdx.z*d; j < y-d; j += blockDim.z) {
        for(size_t k = threadIdx.y*d; k < z-d; k += blockDim.y) {
            value_type sum_local = 0;
            for(size_t di = i-d; di < i+d+1; di++) {
                for(size_t dj = j-d; dj < j+d+1; dj++) {
                    for(size_t dk = k-d; dk < k+d+1; dk++) {
                        sum_local += data[dk + dj*z + di*z*y];
                    }
                }
            }
            data_o[k + j*z + i*z*y] = sum_local;
        }
    }
}
```

Running the raw pointer variant with unsigned vs `size_t` as the loop indices, results in a timing of 31ms vs 56ms. The same is observed for the `mdspan` variant when switching in the `mdspan` implementation the `size_type` from `size_t` to unsigned. The 31ms result can also be obtained when leaving `size_type` as `size_t` but casting `extents.extent(r)` to the user provided index type inside the *layout mappings* index calculation operator while using unsigned as the loop index type in the algorithm.

§ 2.2 Possible Ways To Address The Issue

§ 2.2.1 Mappings Doing Offset Calculation With Argument Type

One way to address this issue would be for mappings to do all their internal calculations with the `common_type` of the user-provided indices. That includes casting `extents.extent(i)`. However the drawback of this approach is that it is hard to identify overflows, which depend on layout as well.

```
// The following is ok, extents converts to size_t, required_span_size  
returns size_t too  
mdspan<double, dextents<3>, layout_right> r(ptr, 2000, 2000, 2000);  
mdspan<double, dextents<3>, layout_left> l(ptr, 2000, 2000, 2000);  
...  
r(1, 1, 1000) = 5; // ok  
r(1000, 1, 1) = 5; // overflow  
l(1, 1, 1000) = 5; // overflow  
l(1000, 1, 1) = 5; // ok
```

In particular, in situations where allocations and `mdspan` creation happens in another code, location this could be an issue.

§ 2.2.2 Template extents on size_type

In order to make overflow a better controllable artifact, and avoid accidental overflows we can make the index type part of the type. The natural place for this is `extents`. Every index calculation related piece of the `mdspan` proposal gets its `size_type` from `extents`, specifically both layout mappings, and `mdspan` itself is required to get its public `size_type` type member from `extents_type::size_type`. Furthermore, `extents` defines the complete iteration space for `mdspan`. Note, that a mapping might require a larger integer range than the product of extents (e.g. `layout_stride::required_span_size` can return a number larger than the product of its extents).

§ 2.2.3 Second Extents Type Templated on size_type

Instead of modifying `extents`, we could introduce a new type `basic_extents` which is templated on the size type and the extents, but otherwise is identical to `extents`: When we can make anything in the `mdspan` proposal which accepts `extents` also accept `basic_extents`.

Potentially, `extents` could be just a template alias to `basic_extents`:

```
template<size_t ... Extents>  
using extents = basic_extents<size_t, Extents...>;
```

Unfortunately that means that the following type of code would not work:

```
template<class T, class L, class A, size_t ... Extents>  
void foo(mdspan<T, extents<Extents...>, L, A> a) {...{
```

However we believe the common use case would be to template on the `extents` object itself, mitigating this issue:

```
template<class T, class E, class L, class A>
void foo(mdspan<T,E,L,A> a) {...}
```

§ 2.2.4 Template LayoutPolicy on size_type

We could also template the layout policy class on `size_type`, and use that type for the offset calculation, casting `extents::extent` explicitly on use. However this still means that the size of the object is larger (i.e. we would still store 64bit extents, instead of 32bit) and that additional casts will happen.

§ 2.2.5 What we prefer:

All in all we prefer the option of making extents require the additional argument (2.2.2), with the next best thing being the introduction `basic_extents` and making extents an alias to `basic_extents` with `size_t` as the `size_type`. If LEWG would prefer the second option, the wording is largely the same with the following changes at the end:

- Rename extents to `basic_extents` throughout P0009 and
- Add an alias in `[mdspan.syn]`:

```
template<size_t ... Extents>
using extents = basic_extents<size_t, Extents...>;
```

LEWG would need to decide whether to make `dextents` have a `size_type` template parameter or not.

§ 2.3 Why we can't fix this later

In principle we could add a second extents type later, though it may break code such as the one shown before (in the sense that it wouldn't generally work for every instance of `mdspan` anymore):

```
template<class T, class L, class A, size_t ... Extents>
void foo(mdspan<T,extents<Extents...>,L,A> a) {...}
```

§ 3 Editing Notes

The proposed changes are relative to the working draft of the standard as of [P0009](#) R16.

§ 4 Wording

§ 4.1 In 22.7.X `[mdspan.syn]`

Replace:

```
template<size_t... Extents>
class extents;
```

```
template<size_t Rank>
using extents = see below;
```

with:

```
template<class SizeT, size_t... Extents>
class extents;

template<class SizeT, size_t Rank>
using extents = see below;
```

§ 4.2 In 22.7.X.1 [mdspan.extents.overview]:

— In the synopsis replace:

```
template<size_t... Extents>
class extents {
public:
    using size_type = size_t;
    using rank_type = size_t;
```

with

```
template<class SizeT, size_t... Extents>
class extents {
public:
    using size_type = SizeT;
    using rank_type = size_t;
```

— In the synopsis replace:

```
static constexpr size_type static_extent(rank_type) noexcept;
```

with

```
static constexpr size_t static_extent(rank_type) noexcept;
```

— In the synopsis replace:

```
template<size_t... OtherExtents>
explicit(see below)
constexpr extents(const extents<OtherExtents...>&) noexcept;
```

with:

```
template<class OtherSizeT, size_t... OtherExtents>
explicit(see below)
constexpr extents(const extents<OtherSizeT, OtherExtents...>&)
noexcept;
```

— In the synopsis replace:

```
// [mdspan.extents.cmp], extents comparison operators
template<size_t... OtherExtents>
```

```
friend constexpr bool operator==(const extents&, const
    extents<OtherExtents...>&) noexcept;
```

with:

```
// [mdspan.extents.cmp], extents comparison operators
template<class OtherSizeT, size_t... OtherExtents>
    friend constexpr bool operator==(const extents&, const
        extents<OtherSizeT, OtherExtents...>&) noexcept;
```

— In the synopsis add the following public static exposition only member functions:

```
template<class... Integrals>
static constexpr bool are-valid-extents(Integrals... values) noexcept;
    // exposition only

template<class OtherSizeT, size_t... OtherExtents>
static constexpr bool are-valid-extents(extents<OtherSizeT,
    OtherExtents>) noexcept; // exposition only

template<class OtherSizeT, size_t N>
static constexpr bool are-valid-extents(array<OtherSizeT, N>) noexcept;
    // exposition only
```

— After paragraph 2 insert a new paragraph 3:

If `SizeT` is not an integral type other than `bool`, then the program is ill-formed.

§ 4.3 In 22.7.X.2 [mdspan.extents.helpers]:

— Add new paragraphs after 8:

```
template<class... Integrals>
static constexpr bool are-valid-extents(Integrals... values) noexcept; //
    exposition only
```

Returns:

- true if `sizeof...(Integrals) == 0` is true, otherwise
- true if `((values > 0) && ...)` is true and each element of `values` is a representable value of `size_type`, otherwise
- false.

```
template<class OtherSizeT, size_t... OtherExtents>
static constexpr bool are-valid-extents(extents<OtherSizeT, OtherExtents>
    e) noexcept; // exposition only
```

Returns:

- true if `sizeof...(OtherExtents) == 0` is true, otherwise
- true if `e.extent(r) > 0` is true for all rank index `r` of `e` and `e.extent(r)` is a representable value of `size_type` for all rank index `r` of `e`, otherwise

— false.

```
template<class OtherSizeT, size_t N>
static constexpr bool are-valid-extents(array<OtherSizeT, N> arr)
    noexcept; // exposition only
```

Returns:

— true if $N == 0$ is true, otherwise

— true if for all r in the range of $[0, N)$ $\text{arr}[r] > 0$ is true and $\text{arr}[r]$ is a representable value of `size_type`, otherwise

— false.

§ 4.4 In subsection 22.7.X.3 [mdspan.extents.cons]

— Change the following:

```
template<size_t... OtherExtents>
explicit((((Extents!=dynamic_extent) && (OtherExtents==dynamic_extent))
|| ... ))
constexpr extents(const extents<OtherExtents...>& other) noexcept;
```

to:

```
template<class OtherSizeT, size_t... OtherExtents>
explicit(see below)
constexpr extents(const extents<OtherSizeT, OtherExtents...>& other)
    noexcept;
```

— Change Paragraph 2 to:

Preconditions:

— `other.extent(r)` equals E_r for each r for which E_r is a static extent, and

— `are-valid-extents(other)` is true.

— Add new paragraph 4:

Remarks: The expression inside `explicit` is equivalent to:

```
((((Extents!=dynamic_extent) && (OtherExtents==dynamic_extent)) ||
... )) ||
(numeric_limits<size_type>::max() <
 numeric_limits<OtherSizeT>::max())
```

— Change Paragraph 6 to:

Preconditions:

— If `sizeof...(SizeTypes) != rank_dynamic()` is true, `exts_arr[r]` equals E_r for each r for which E_r is a static extent, and

— *are-valid-extents*(exts...) is true.

— Change Paragraph 9 to:

Preconditions:

— If $N \neq \text{rank_dynamic}()$ is true, $\text{exts}[r]$ equals E_r for each r for which E_r is a static extent, and

— *are-valid-extents*(exts) is true.

— Change paragraph 12 to:

Remarks: The deduced type is `dextents<size_t, sizeof...(Integrals)>`.

§ 4.5 In subsection 22.7.X.4 [mdspan.extents.obs]

— Replace the following:

```
static constexpr size_type static_extent(rank_type i) const
    noexcept;
```

with:

```
static constexpr size_t static_extent(rank_type i) const noexcept;
```

§ 4.6 In subsection 22.7.X.5 [mdspan.extents.compare]

— Replace the following:

```
template<size_t... OtherExtents>
    friend constexpr bool operator==(const extents& lhs,
                                     const extents<OtherExtents...>&
                                     rhs) noexcept;
```

with:

```
template<class OtherSizeT, size_t... OtherExtents>
    friend constexpr bool operator==(const extents& lhs,
                                     const extents<OtherSizeT,
                                     OtherExtents...>& rhs) noexcept;
```

— Change paragraph 1 to:

Returns: true if `lhs.rank() == rhs.rank()` is true and `lhs.extents(r)` equals `rhs.extents(r)` for every rank index `r` of `rhs`, otherwise false.

§ 4.7 In subsection 22.7.X.6 [mdspan.extents.dextents]

— Replace section with:

```
template <class SizeT, size_t Rank>
    using dextents = see below;
```

- 1 *Result:* A type `E` that is a specialization of `extents` such that `E::rank() == Rank` && `E::rank() == E::rank_dynamic()` is true and `E::size_type` denotes `SizeT`.

§ 4.8 In subsection 22.7.X.3.1 [`mdspan.layout.left.ctors`]

- After paragraph 3 add new paragraph:

Preconditions: `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

- After paragraph 5 add new paragraph:

Preconditions: `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

- Change paragraph 8 to:

Preconditions:

- If `extents_type::rank() > 0` is true then for all r in the range $[0, \text{extents_type::rank}())$, `other.stride(r)` equals `extents().fwd-prod-of-extents(r)`, and
- `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

§ 4.9 In subsection 22.7.X.4.1 [`mdspana.layout.right.ctors`]

- After paragraph 3 add new paragraph:

Preconditions: `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

- After paragraph 5 add new paragraph:

Preconditions: `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

- Change paragraph 8 to:

Preconditions:

- If `extents_type::rank() > 0` is true then for all r in the range $[0, \text{extents_type::rank}())$, `other.stride(r)` equals `extents().rev-prod-of-extents(r+1)`, and
- `other.required_span_size()` is a representable value of `size_type` ([`basic.fundamental`]).

§ 4.10 In Subsection 22.7.X.5.2 [mdspan.layout.stride.cotrs]

— add in paragraph 6 additional precondition:

— `other.stride(r) > 0` is true for all rank index `r` of `extents()`,

§ 4.11 In subsection 22.7.X.1 [mdspan.mdspan.overview]

— In the synopsis replace:

```
static constexpr size_type static_extent(size_t r) { return
    Extents::static_extent(r); }
```

with:

```
static constexpr size_t static_extent(size_t r) { return
    Extents::static_extent(r); }
```

— In the synopsis replace:

```
constexpr size_type size() const;
```

with

```
constexpr size_t size() const;
```

— In the synopsis replace:

```
template <class ElementType, class SizeType, size_t N>
mdspan(ElementType*, const array<SizeType, N>&)
    -> mdspan<ElementType, dextents<N>>;

template <class ElementType, size_t... ExtentsPack>
mdspan(ElementType*, const extents<ExtentsPack...>&)
    -> mdspan<ElementType, extents<ExtentsPack...>>;
```

with:

```
template <class ElementType, class SizeType, size_t N>
mdspan(ElementType*, const array<SizeType, N>&)
    -> mdspan<ElementType, dextents<size_t, N>>;

template <class ElementType, class SizeT, size_t... ExtentsPack>
mdspan(ElementType*, const extents<SizeT, ExtentsPack...>&)
    -> mdspan<ElementType, extents<SizeT, ExtentsPack...>>;
```

§ 4.12 In subsection 22.7.X.1 [mdspan.mdspan.ctors]

— Change code after paragraph 19 to:

```
template <class ElementType, class SizeType, size_t N>
mdspan(ElementType*, const array<SizeType, N>&)
    -> mdspan<ElementType, dextents<size_t, N>>;
```

§ 4.13 In Subsection 22.7.X [mdspan.submdspan]

— Add a sub-bullet in 11.1:

— `typename SubExtents::size_type` is `typename Extents::size_type`

§ 5 Implementation

There is an mdspan implementation available at <https://github.com/kokkos/mdspan/>.

§ 6 Related Work

— **P0009** : mdspan